

Workshop 01 – Besprechung der Lösung

Musterlösung: `loesung.py`, Variante mit TensorBoard: `loesung_tensorboard.py`.

1. Daten laden und vorbereiten

```
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0
```

CIFAR-10 kommt bereits in Trainings- (50.000) und Testdaten (10.000) aufgeteilt. Die Pixelwerte liegen als `uint8` (0–255) vor; die Division bringt sie auf 0–1, was das Training numerisch stabiler macht. Die Labels bleiben Integer (0–9) – darum später `sparse_categorical_crossentropy`.

2. Modell

```
model = Sequential([
    Conv2D(32, (3, 3), activation="relu", input_shape=(32, 32, 3)), # -> 30x30x32
    MaxPooling2D(2, 2), # -> 15x15x32
    Conv2D(64, (3, 3), activation="relu"), # -> 13x13x64
    MaxPooling2D(2, 2), # -> 6x6x64
    Flatten(), # -> 2304
    Dense(64, activation="relu"),
    Dense(10, activation="softmax"),
])
```

- Zwei Conv-Blöcke mit steigender Filterzahl (32 → 64): frühe Schichten lernen Kanten und Farben, spätere komplexere Muster.
- `MaxPooling2D(2, 2)` halbiert die räumliche Auflösung und macht das Netz robuster gegen kleine Verschiebungen.
- `Flatten` überführt den 6×6×64-Tensor in einen Vektor, danach zwei Dense-Layer; der letzte hat 10 Neuronen (eine pro Klasse) mit `softmax`.

`model.summary()` zeigt ca. 167.000 trainierbare Parameter.

3. Kompilieren und trainieren

```
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
history = model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))
```

Adam ist ein robuster Standard-Optimizer. Die Testdaten werden hier als Validierung genutzt – für einen Workshop ausreichend, in der Praxis würde man einen separaten Validierungssplit vom Trainingsset abzweigen.

4. Trainingsverlauf

`history.history` enthält je Epoche `loss`, `accuracy`, `val_loss` und `val_accuracy`. Die Lösung plottet Accuracy und Loss nebeneinander.

Wichtig: `plt.savefig()` muss **vor** `plt.show()` aufgerufen werden – `show()` leert die Figur, danach wird eine weiße Grafik gespeichert.

5. Ergebnis

Nach 10 Epochen liegt die Test-Accuracy typischerweise bei **ca. 68–72 %**. Im Plot ist erkennbar, dass `accuracy` weiter steigt, während `val_accuracy` ab etwa Epoche 5–6 stagniert und `val_loss` wieder zunimmt: klassisches **Overfitting**. Genau hier setzt Workshop 02 an (Dropout, BatchNormalization, Callbacks).

Bonus: TensorBoard

loesung_tensorboard.py ergänzt nur den Callback:

```
tensorboard_callback = TensorBoard(log_dir="logs_cifar_01")  
model.fit(..., callbacks=[tensorboard_callback])
```

Anzeigen mit `tensorboard --logdir logs_cifar_01`.